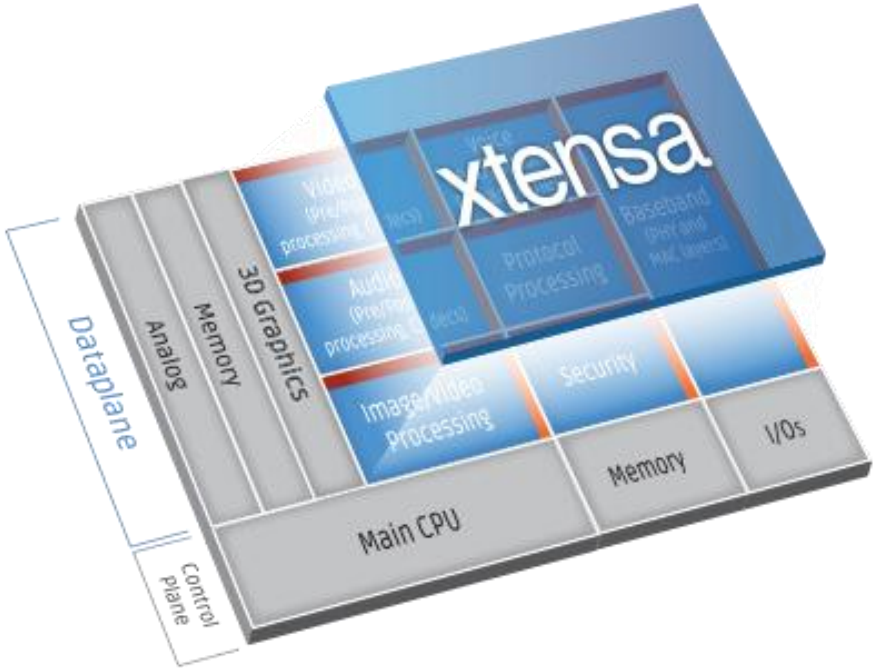
Características del ESP 32

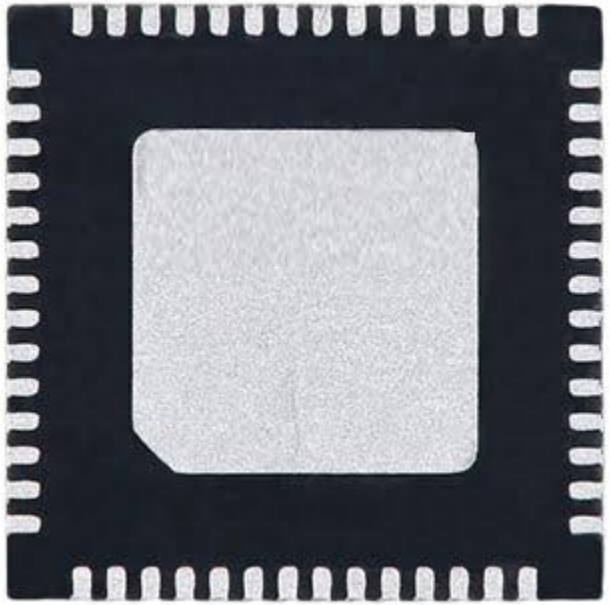
El ESP32 es un microcontrolador muy popular desarrollado por Espressif Systems, diseñado para aplicaciones de Internet de las Cosas (IoT), electrónica embebida y comunicaciones inalámbricas. Este microcontrolador se destaca por su alta integración, eficiencia energética y capacidades de conectividad



El ESP32 cuenta con un procesador dual-core basado en la arquitectura Xtensa LX6 de Tensilica. Cada núcleo puede operar a velocidades de hasta 240 MHz, y se puede asignar a diferentes tareas para maximizar la eficiencia.

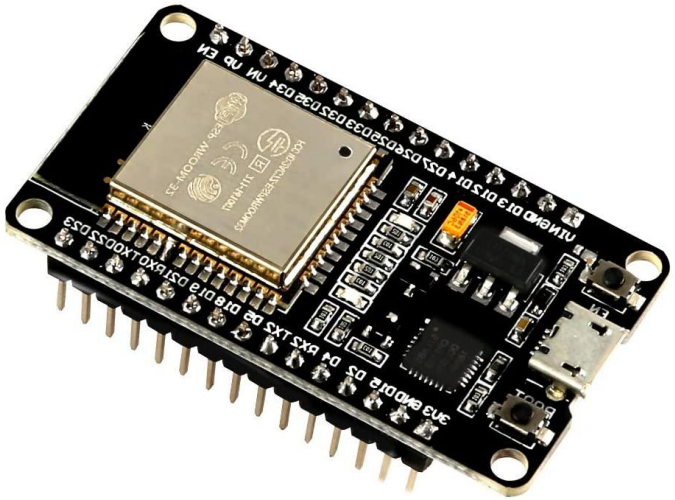






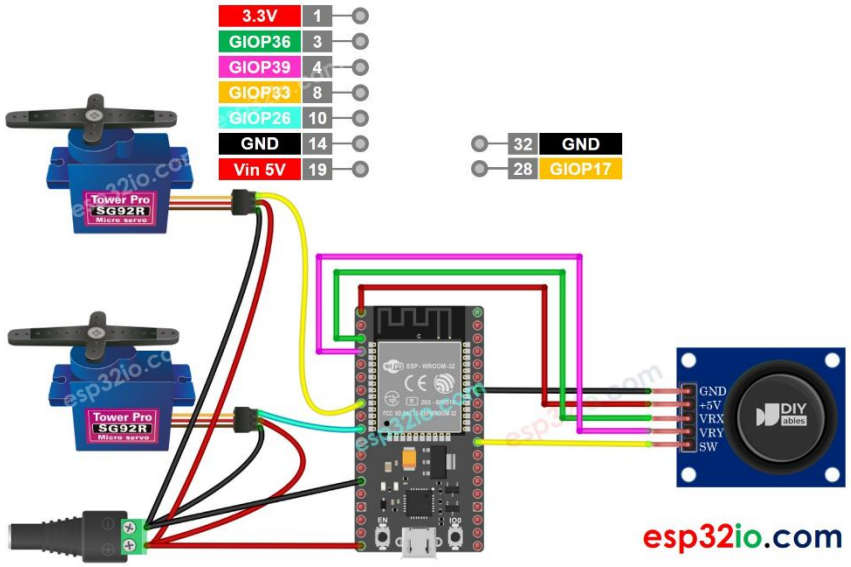


Mientras que una computadora moderna utiliza procesadores como Intel o AMD con arquitecturas x86 o ARM, que son capaces de manejar múltiples hilos de procesamiento a velocidades superiores a los 3 GHz, el ESP32 está optimizado para tareas de baja potencia y operaciones en tiempo real.





Los procesadores de las computadoras están diseñados para ejecutar sistemas operativos complejos, aplicaciones multitarea, y procesamiento intensivo de datos, mientras que el ESP32 está enfocado en tareas específicas como controlar sensores, manejar redes inalámbricas, y realizar operaciones de entrada/salida.

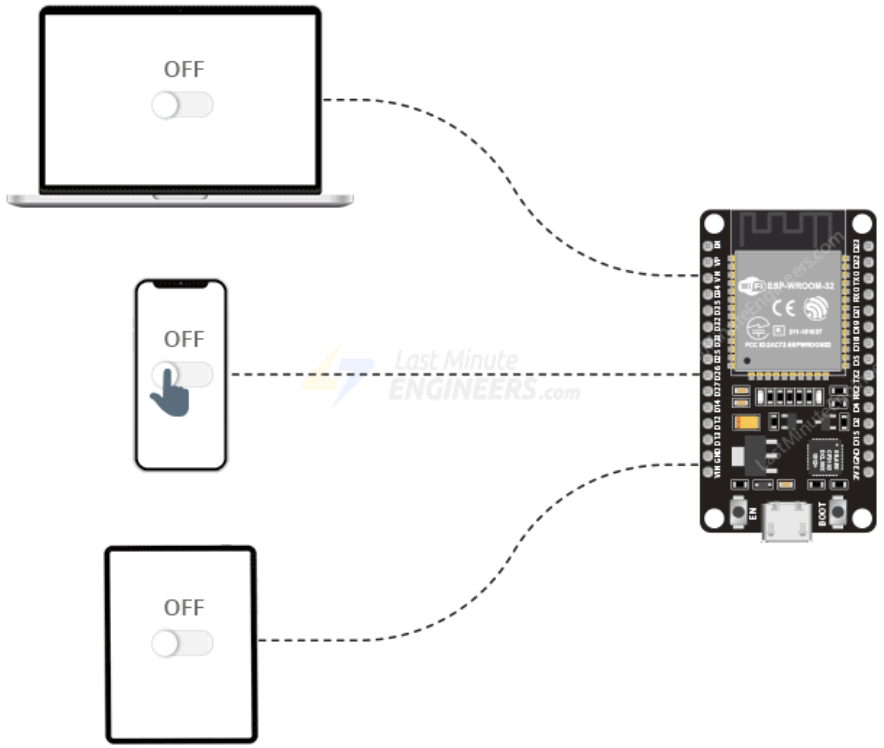


El ESP32

•Wi-Fi: Compatible con 802.11 b/g/n, capaz de operar en las bandas de 2.4 GHz.

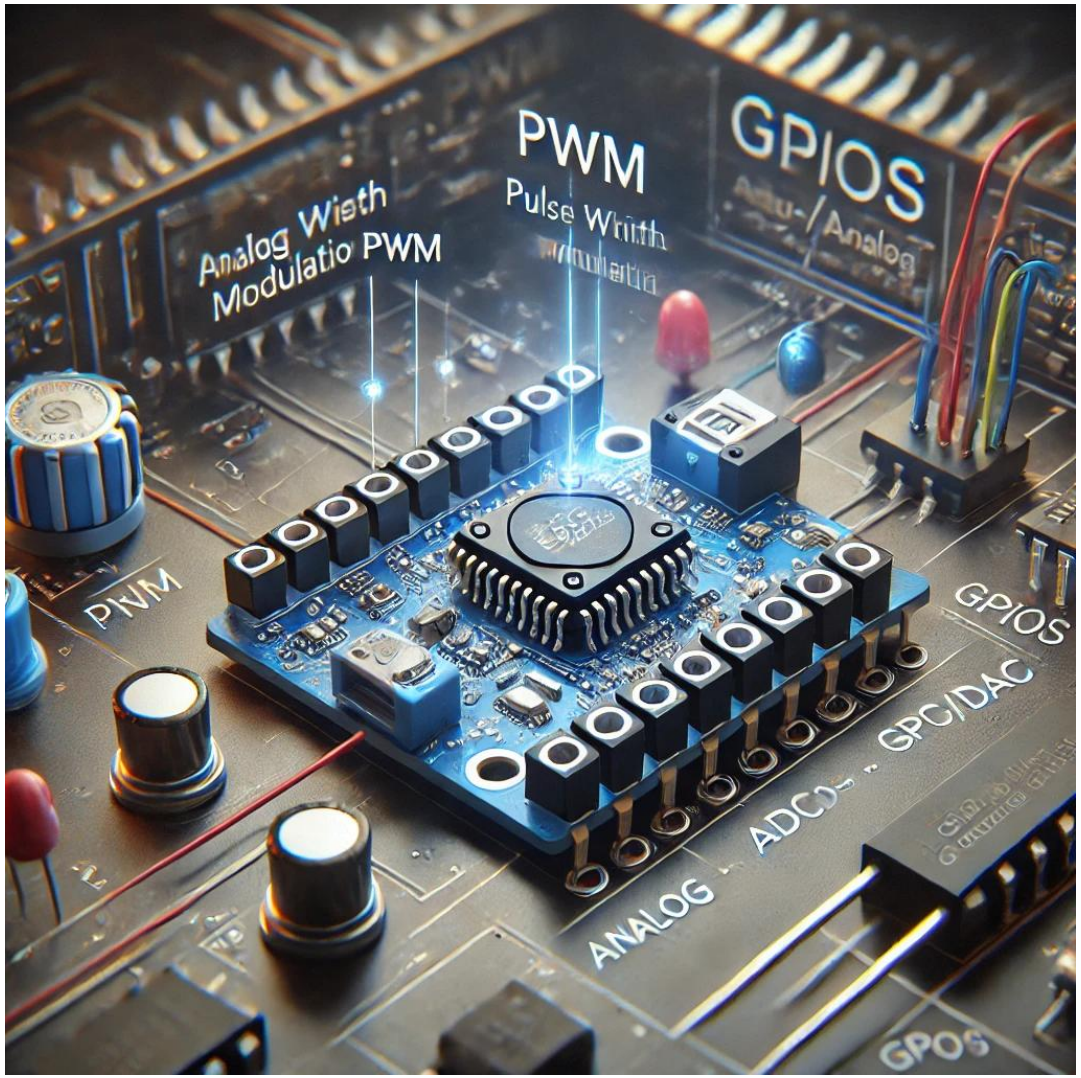
•Bluetooth: Incluye soporte para Bluetooth 4.2 y BLE (Bluetooth Low Energy).

•Interfaces: Incluye UART, SPI, I2C,



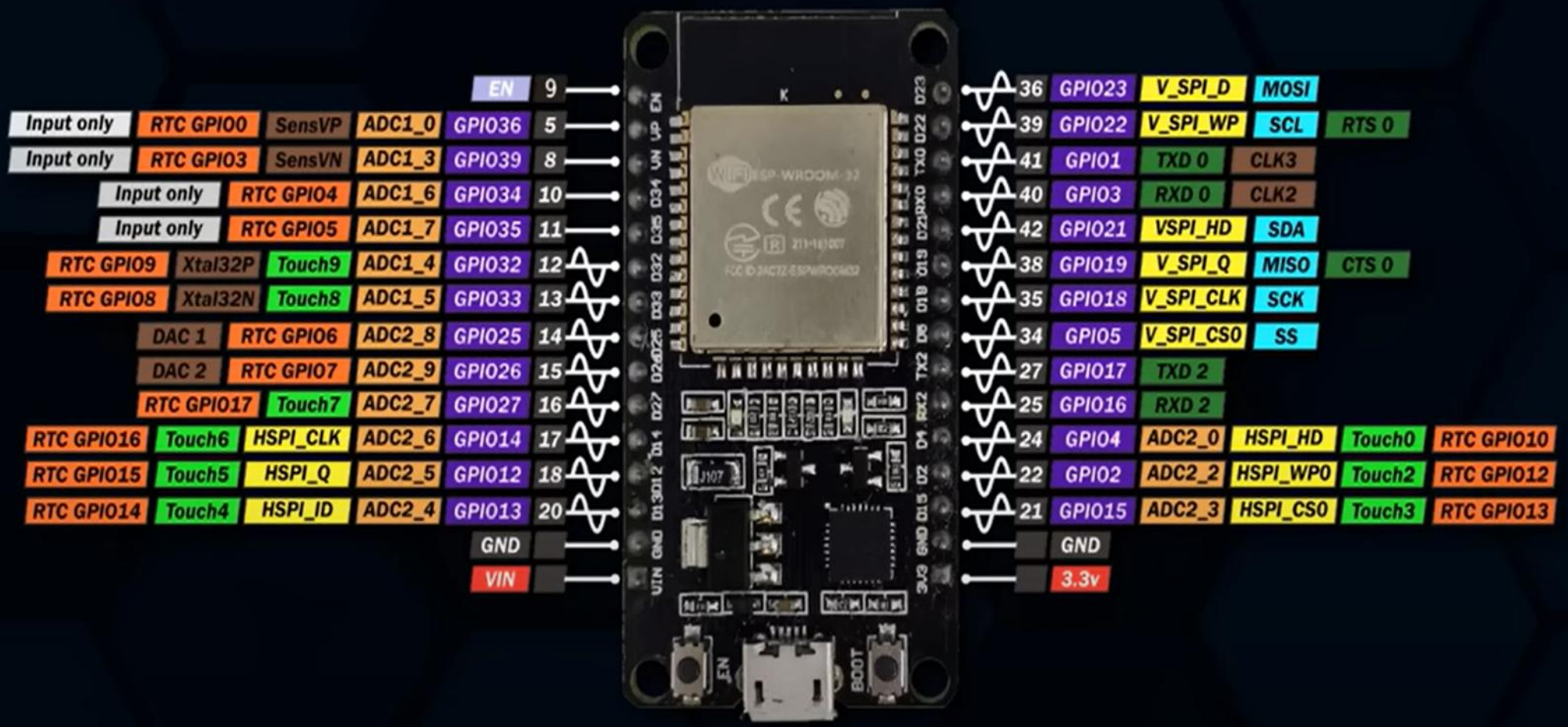
El ESP32 incluye una variedad de periféricos internos:

- **Convertidores ADC/DAC**: Para convertir señales analógicas en digitales y viceversa.
- **Timers**: Para operaciones temporizadas en tiempo real.
- **Controladores de LED y motores**: Mediante PWM (Pulse Width Modulation).



Dispone de 25 pines digitales

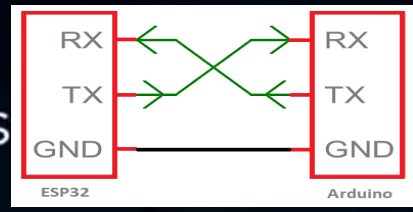
para sensores, leds, botones...





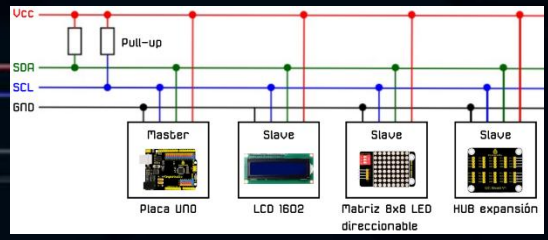
3 Puertos UART (Comunicación serial)

Comunicación asincrona entre dispositivos usando dos pines (TX y RX).



2 Puertos I2S

Transmision de datos de audio.



4 Puertos SPI

- Velocidades de hasta 80 MHz.
- Buffer de 64 bytes para transmisión y recepción.



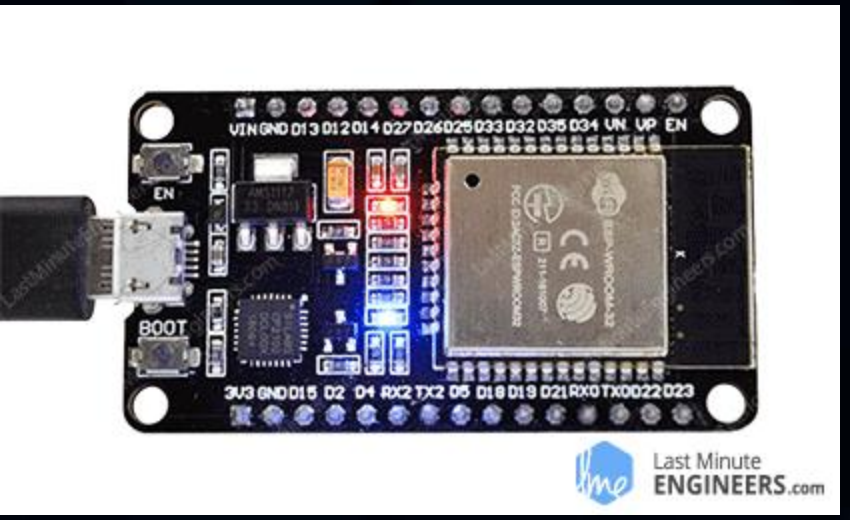
16 pines PWM

Modulación de ancho de pulso.



ALIMENTACIÓN

5v usando el puerto microusb o el pin VIN.



5V



5V

ENTRADAS/SALIDAS

3.3V

El ESP32 puede ser alimentado de varias maneras, principalmente a través del puerto micro-USB o un pin VIN con una fuente de alimentación externa. El voltaje de funcionamiento del ESP32 está entre 2.2V y 3.6V, pero se puede suministrar 5V a través del puerto USB, y el módulo tiene un regulador de voltaje para mantener 3.3V estables.

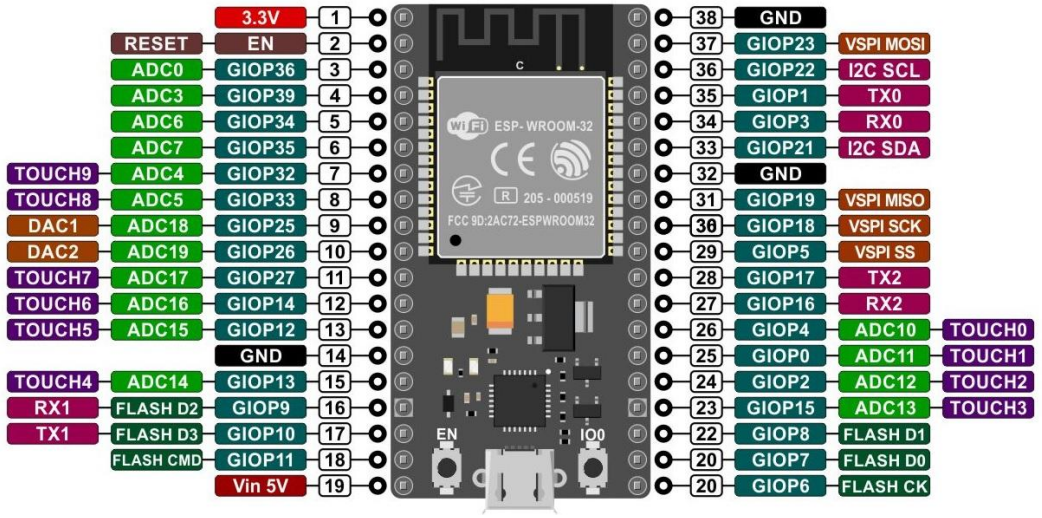
https://esp32io.com/tutorials/how-to-power-esp32#:~:text=You%20can%20use%20the%205v,a%20GND%20pin%20of%20ESP32

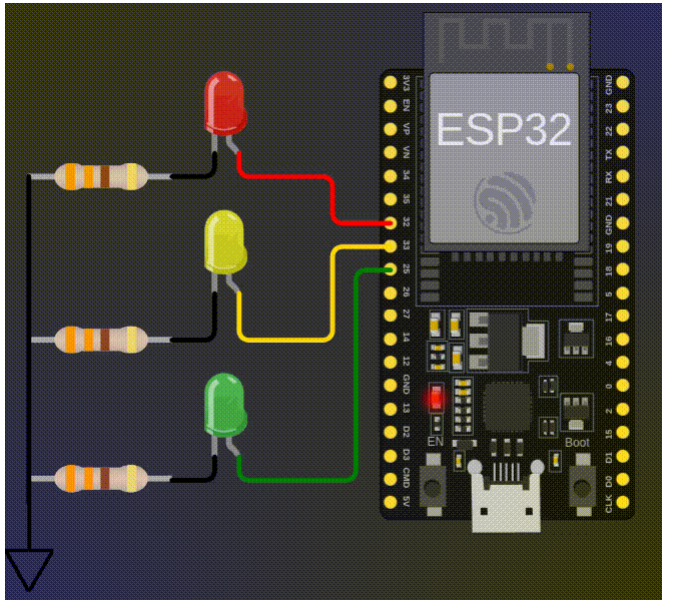
Salidas del ESP 32

Tensiones

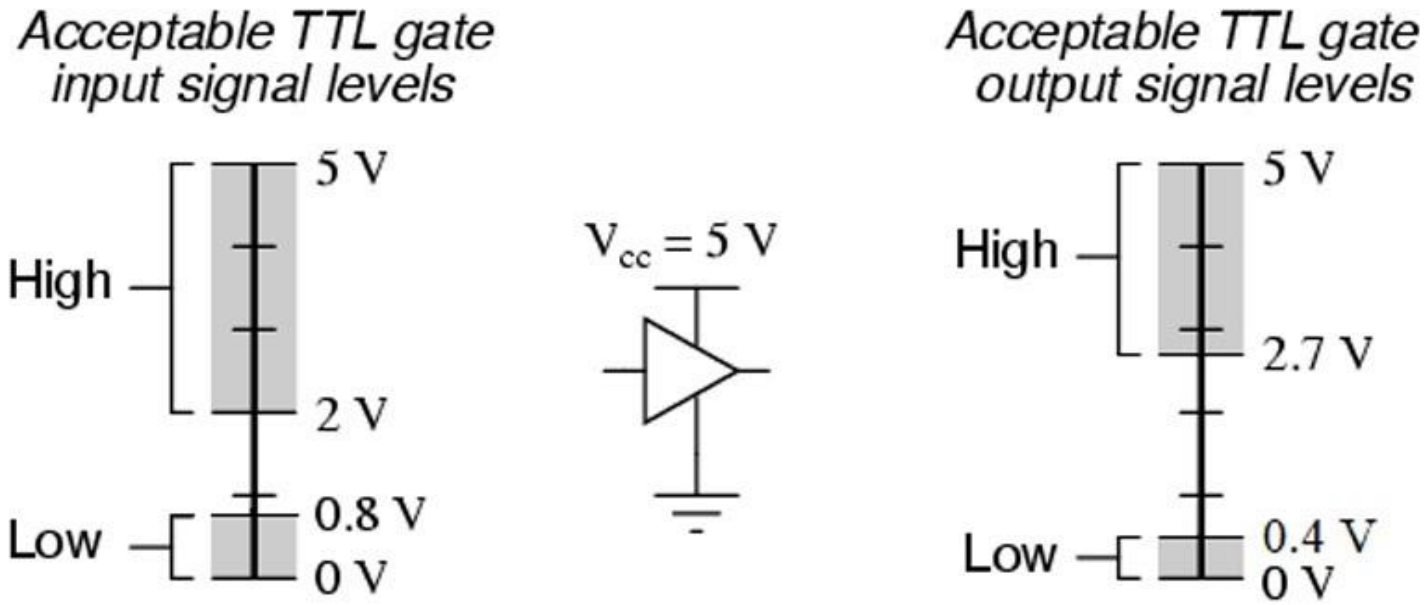
•Tensión de operación (High): Las salidas digitales del ESP32 trabajan a 3.3V. Cuando una salida digital está en estado alto (HIGH), la tensión que entrega es aproximadamente 3.3V.

•Tensión de operación (Low): En estado bajo (LOW), la salida digital proporciona aproximadamente 0V.



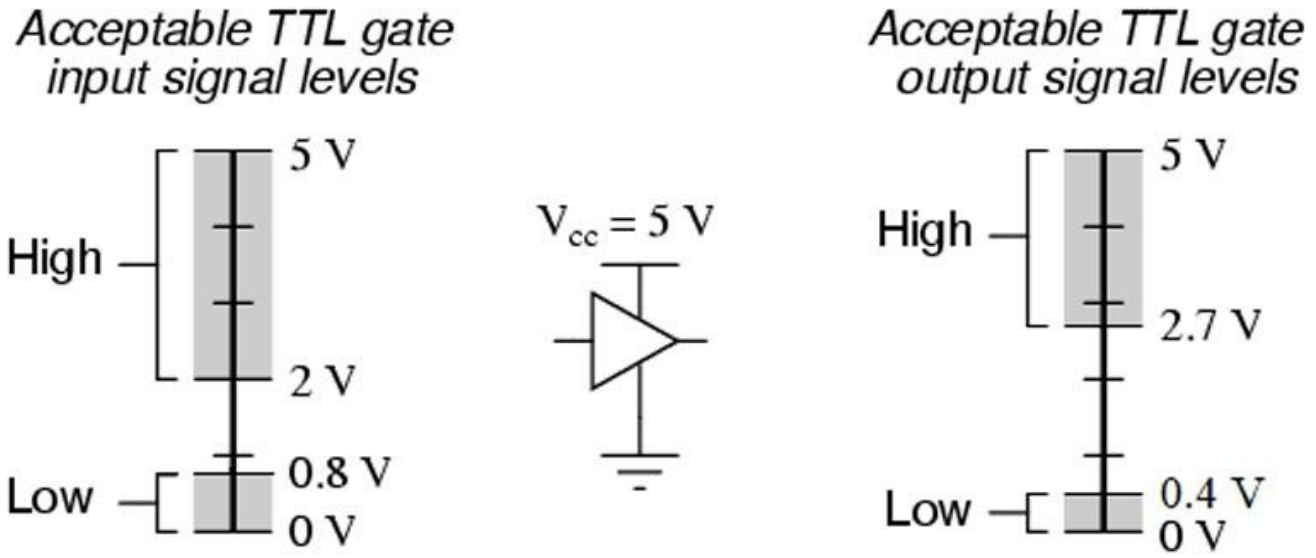


Niveles logicos en electronica digital



La mayoría de los sistemas que utilizamos se basan en niveles TTL de 3,3 V o 5 V. TTL (Transistor-Transistor Logic.)

Niveles logicos en electronica digital



Salida digital está en estado alto (HIGH), la tensión que entrega es aproximadamente 3.3V.

En estado bajo (LOW), la salida digital proporciona aproximadamente 0V.

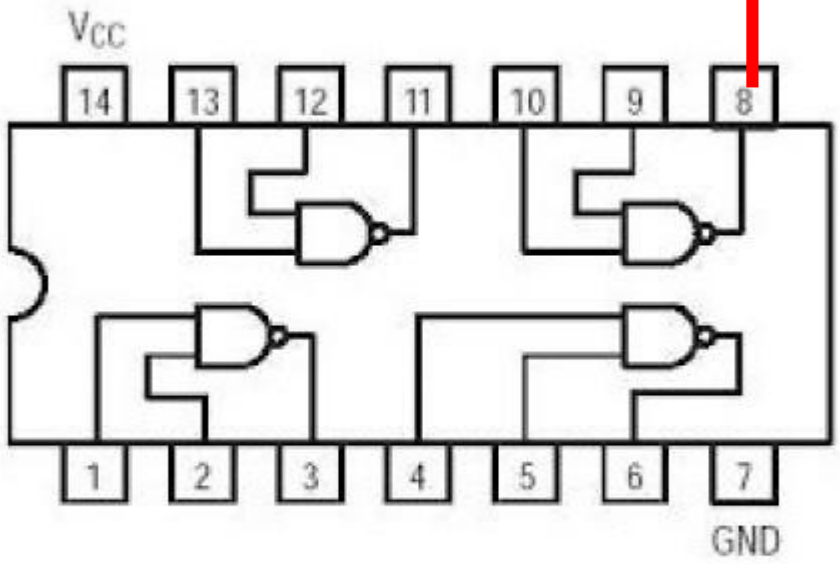
Compatibilidad con TTL (5V)

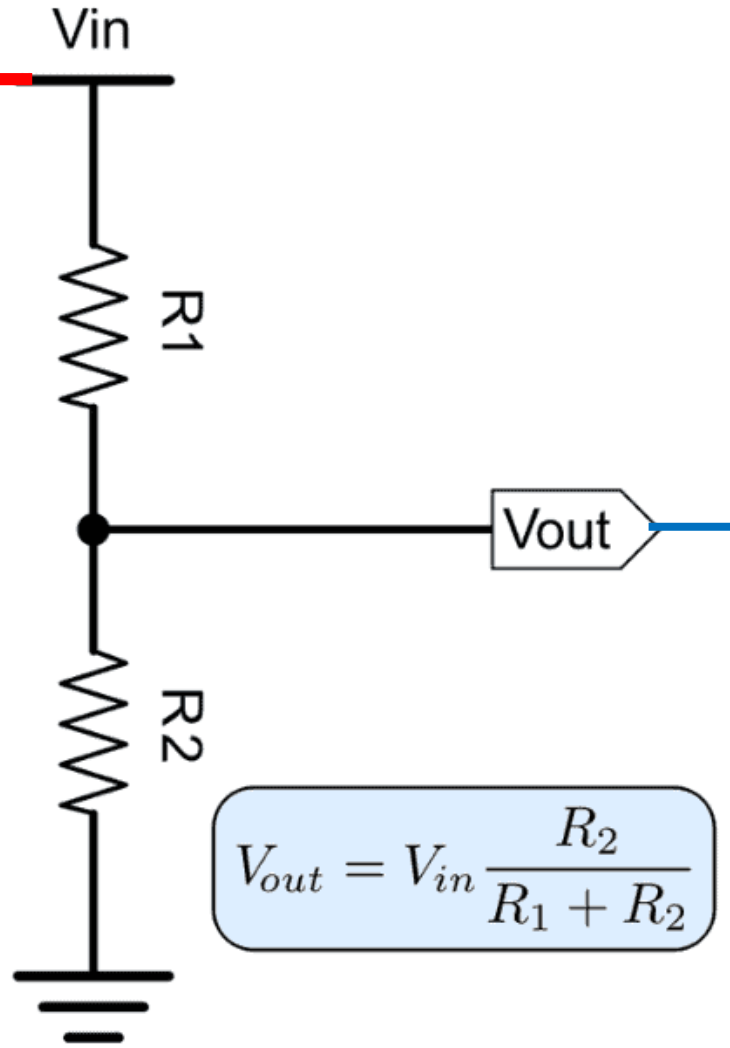
- **ENTRADA (Recepción de señales TTL): El ESP32 puede recibir señales de lógica TTL de 5V en sus pines de entrada, pero no es recomendable aplicar directamente 5V a los pines GPIO, ya que podrían dañarse. Se recomienda usar un divisor de voltaje o un convertidor de nivel para reducir la señal de 5V a 3.3V antes de ingresarla al ESP32.**

- **SALIDA (Envío de señales a dispositivos TTL): El ESP32 puede enviar señales de 3.3V a dispositivos que utilizan lógica TTL. Muchos dispositivos TTL (especialmente los basados en CMOS) pueden reconocer 3.3V como un nivel lógico alto. Sin embargo, si el dispositivo requiere 5V para un "HIGH", podrías necesitar un convertidor de nivel para elevar la señal de 3.3V a 5V.**

Objetivo

Queremos reducir Vin=5V a Vout=3.3V para la entrada del ESP32.

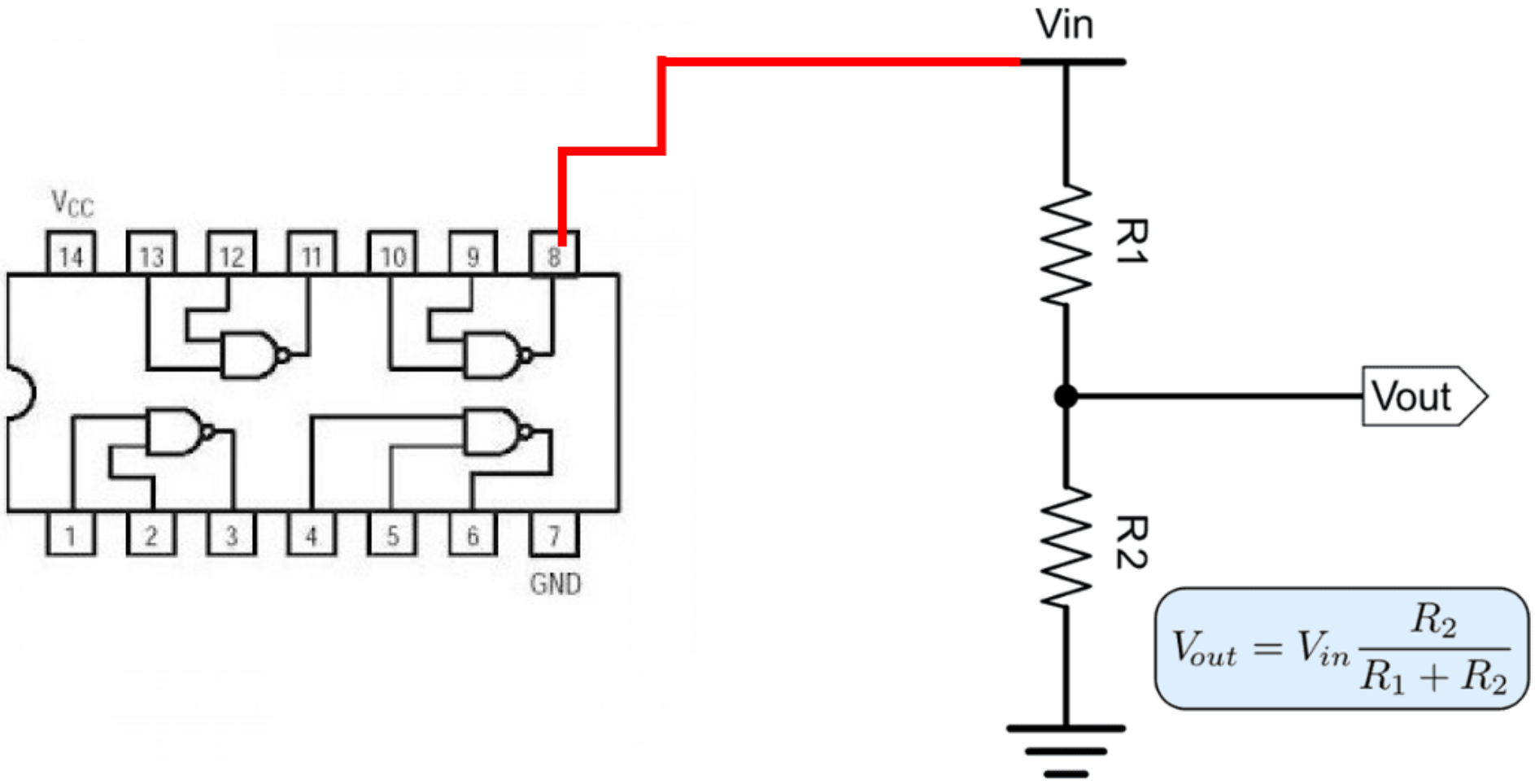






Objetivo

Queremos reducir Vin=5V a Vout=3.3V para la entrada del ESP32.



Objetivo

Queremos reducir Vin=5V a Vout=3.3V para la entrada del ESP32.

$$V_{out} = V_{in} \frac{R_2}{R_1 + R_2}$$

4. Cálculo de las Resistencias

Para obtener 3.3V de una entrada de 5V, podemos reorganizar la fórmula para resolver $R2/(R1 + R2)$:

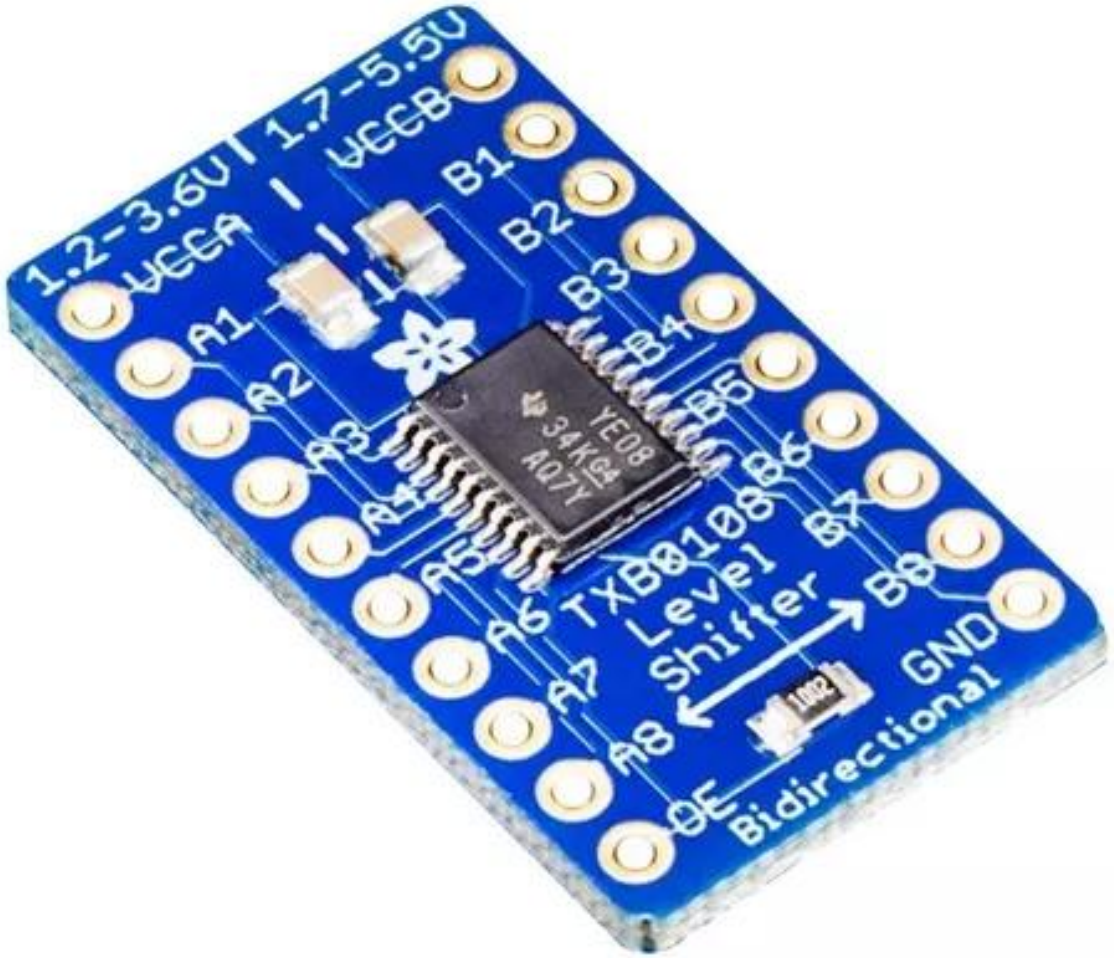
$$\frac{R2}{R1 + R2} = \frac{V_{out}}{V_{in}} = \frac{3.3V}{5V} = 0.66$$

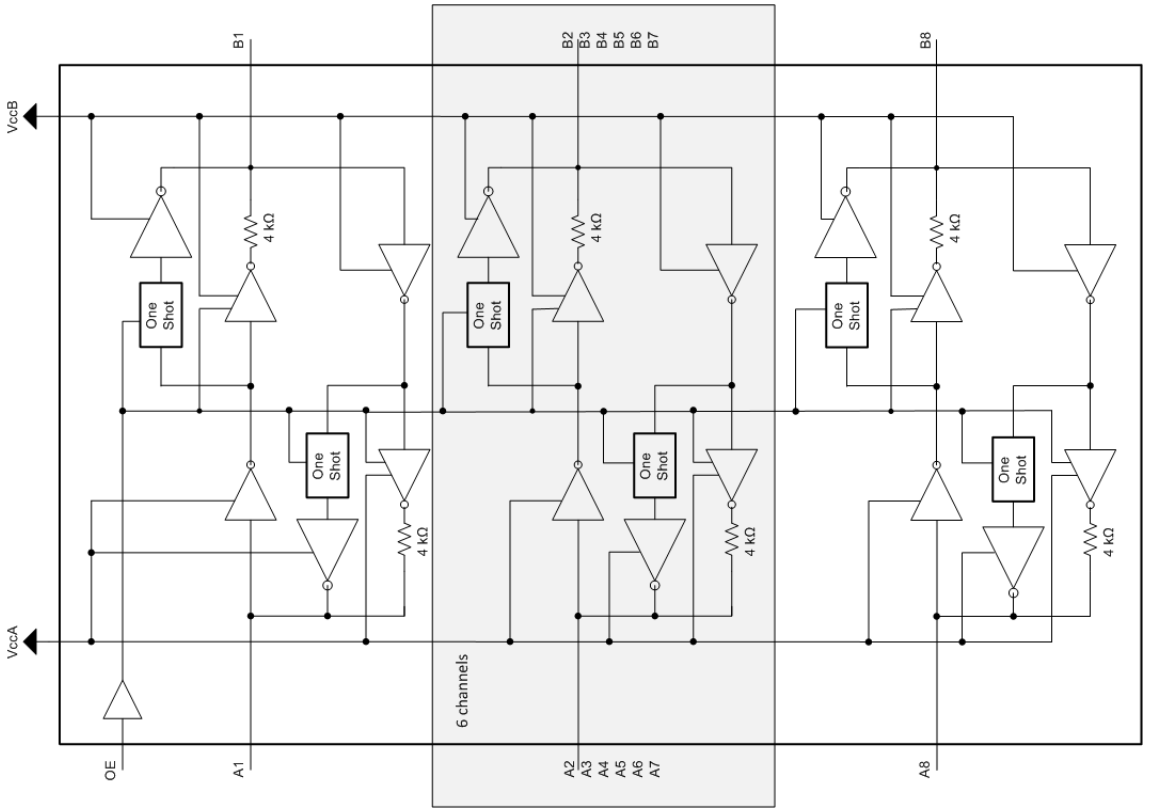
- **R1:** 5.1kΩ
- **R2:** 10kΩ

TXB0108

ACTIVO

Conmutador de nivel de tensión bidireccional de 8 bits con detección de dirección automática y protección

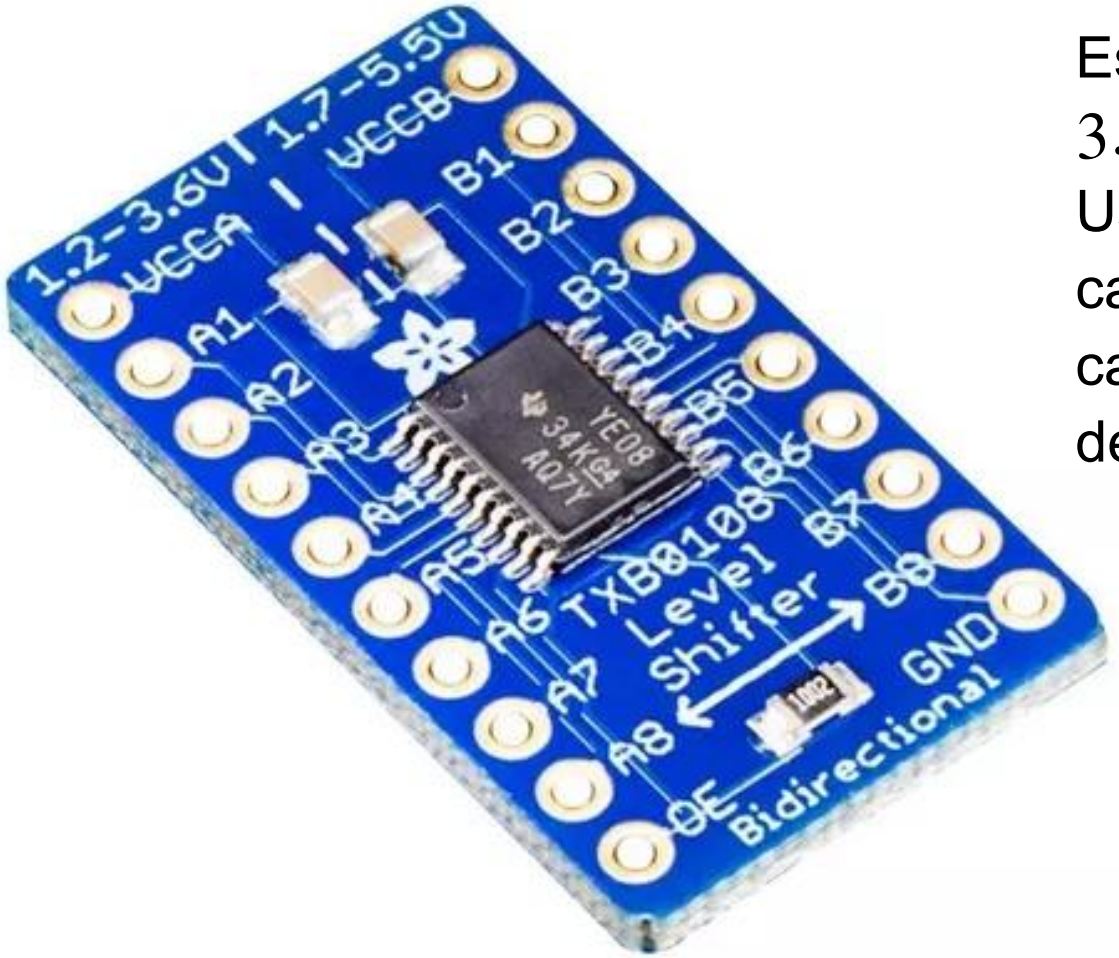




TXB0108

ACTIVO

Conmutador de nivel de tensión bidireccional de 8 bits con detección de dirección automática y protección

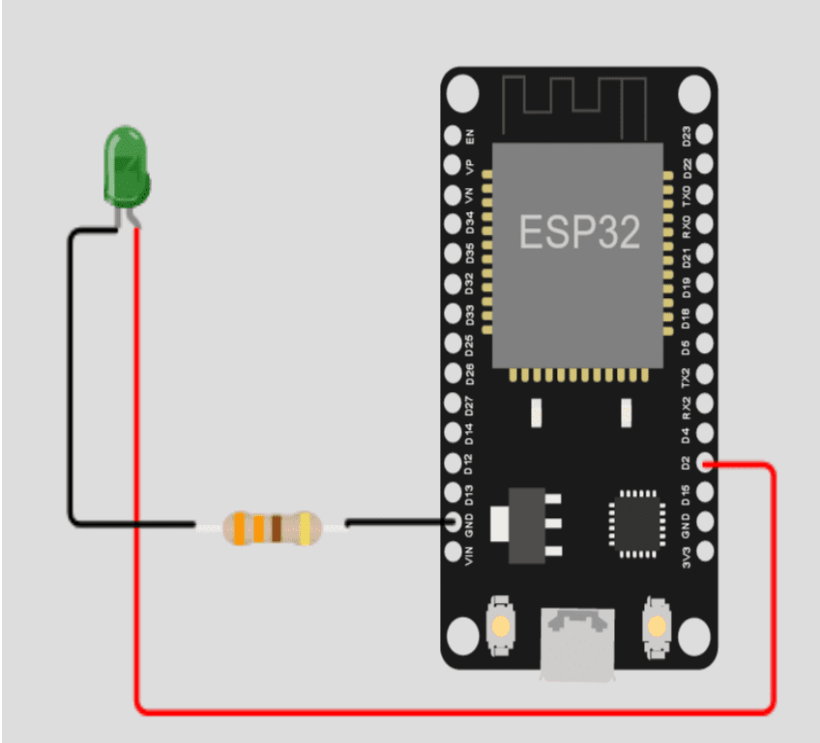


Este convertidor permite convertir niveles, de 3.3 a 5V ó de 5 a 3.3V

Usa el chip TXB0108 y es capaz de realizar cambios de nivel bidireccionales a partir de casi cualquier tensión a cualquier voltaje y detectará automáticamente la dirección.

- **Corriente máxima que puede entregar por pin:**
  - Cada pin de salida digital del ESP32 puede manejar hasta **40 mA**. Sin embargo, para mantener la integridad del microcontrolador y evitar daños a largo plazo, es recomendable no superar los **20 mA** por pin.

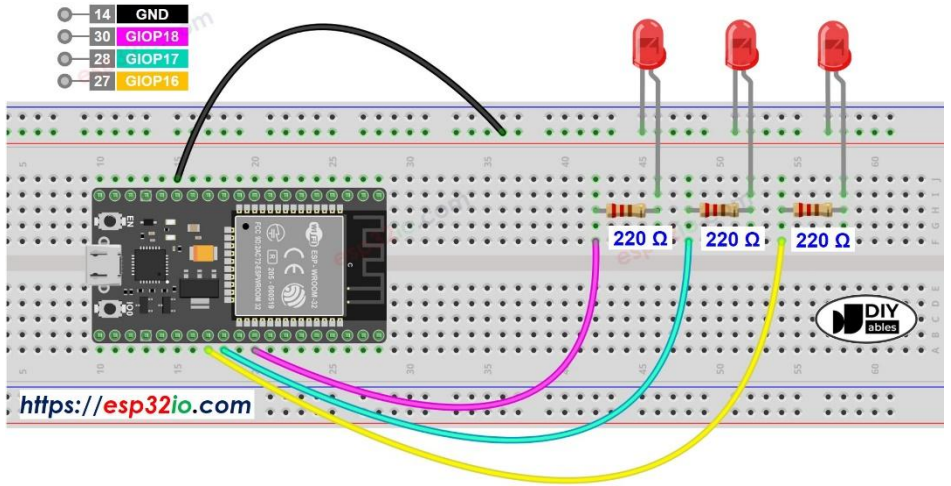
- **Corriente total máxima para el conjunto de pines**:
  - Aunque cada pin individual puede manejar hasta 40 mA, la corriente total suministrada por todos los pines combinados no debe exceder los **120 mA** en conjunto. Es decir, si tienes varios pines activos al mismo tiempo, debes asegurarte de que la suma de las corrientes no supere este valor.



Recomendaciones

- **Uso de resistencias**: Es recomendable utilizar resistencias en serie con las cargas conectadas a los pines digitales para limitar la corriente y proteger tanto el ESP32 como los dispositivos conectados.

- **Cuidado con los dispositivos de alta potencia**: Si necesitas controlar dispositivos que requieren más corriente (como motores, relés grandes, o LEDs de alta potencia), es mejor usar transistores o relés controlados externamente para evitar sobrecargar el ESP32.



Leds

Cómo Funciona el ADC en el ESP32

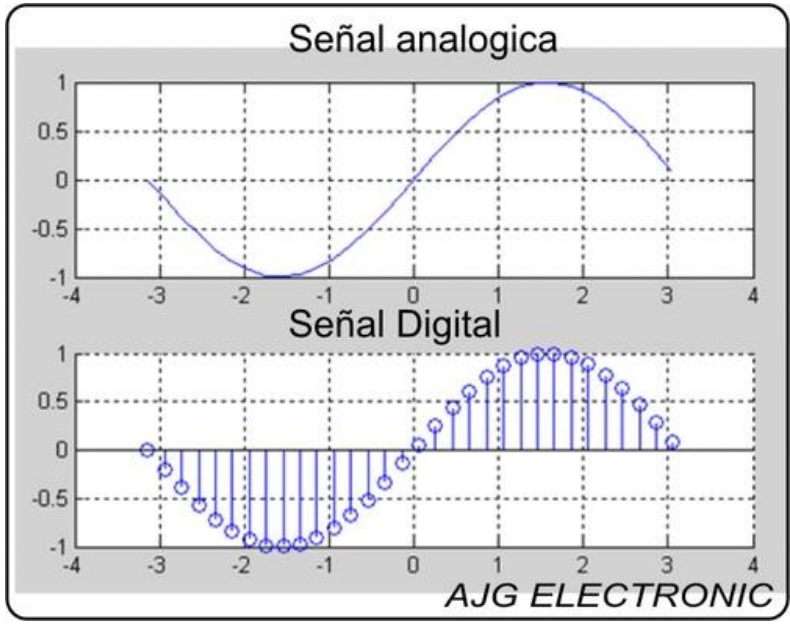
1.Rango de Voltaje:

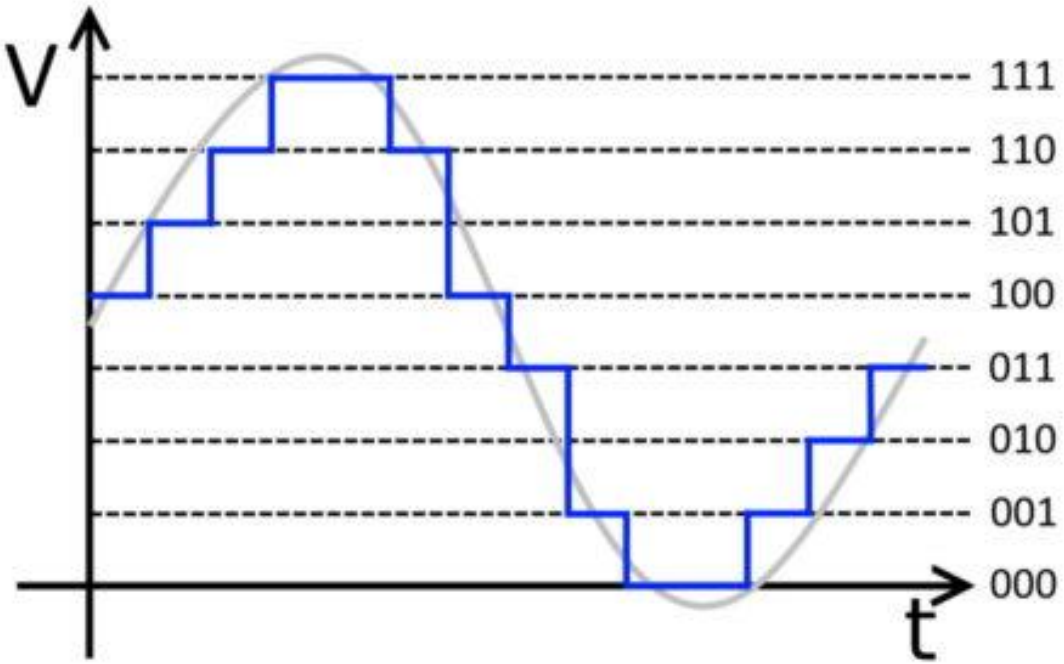
1. El ESP32 tiene un rango de voltaje de entrada para el ADC que varía entre **0V** y **3.3V**.
2. Esto significa que cualquier señal analógica que quieras leer con el ESP32 debe estar dentro de este rango.

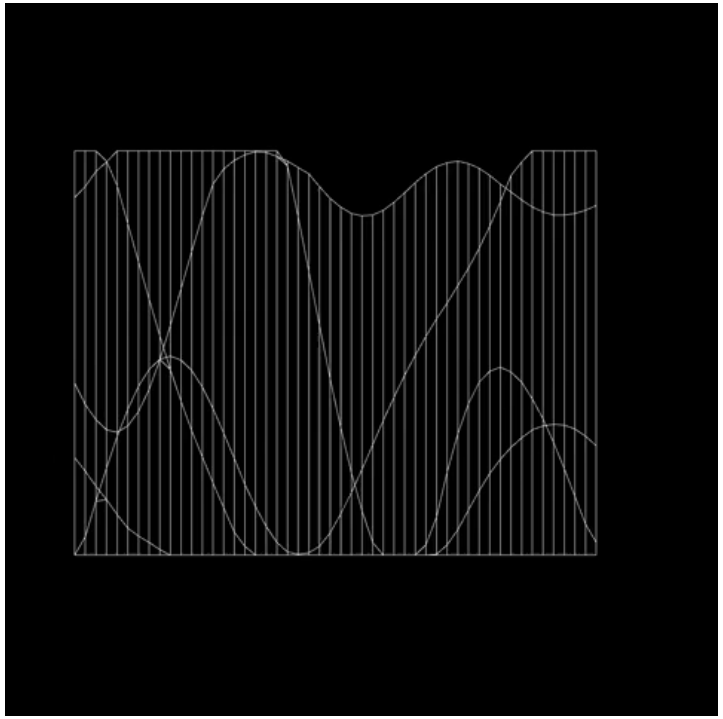
2.Resolución:

1. El ADC del ESP32 tiene una **resolución de 12 bits** por defecto, lo que significa que puede convertir la señal analógica en un valor digital que va de 0 a 4095 (2^12 = 4096 niveles).
2. La resolución puede ser ajustada a valores más bajos si es necesario (9, 10 u 11 bits).

El ADC del ESP32 tiene una **resolución de 12 bits** por defecto, lo que significa que puede convertir la señal analógica en un valor digital que va de 0 a 4095 (2^12 = 4096 niveles).







CONVERSORES D/A

El ESP32 ofrece dos métodos diferentes para generar señales analógicas o simuladas: **PWM (Pulse Width Modulation)** y **DAC (Digital-to-Analog Converter)**. Aunque ambos se pueden utilizar para controlar dispositivos como LEDs, motores o altavoces, funcionan de manera muy diferente y tienen aplicaciones distintas.

DAC (Digital-to-Analog Converter).

El ESP32 tiene dos convertidores digital-analógicos (DAC) de 8 bits, cada uno con una resolución de 256 niveles. Estos DACs permiten generar señales analógicas a partir de valores digitales, con un rango de salida de 0 a 3.3V.



2. Modulación por Ancho de Pulso (PWM)

•Cómo Funciona:

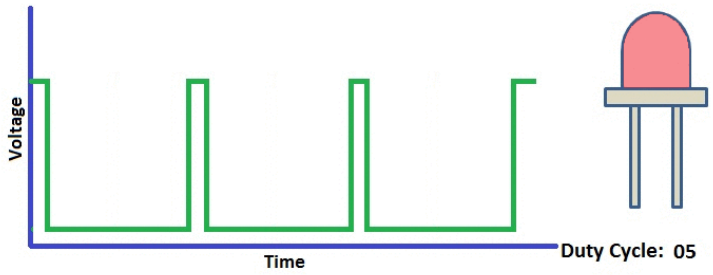
- PWM no genera un voltaje analógico directo, sino que emite una señal digital que alterna rápidamente entre alto (3.3V) y bajo (0V).
- El porcentaje de tiempo que la señal está en estado alto durante un ciclo se llama **ciclo de trabajo**. Al variar el ciclo de trabajo, puedes controlar la potencia promedio entregada a una carga.

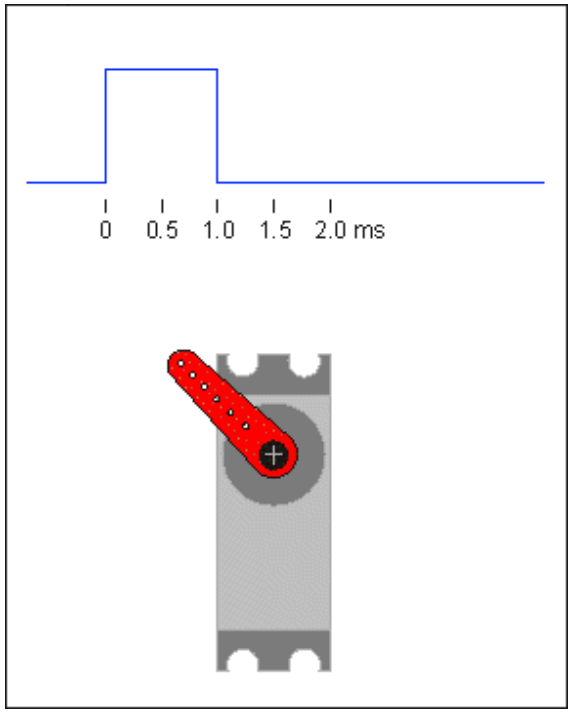
•Salida:

- La señal PWM es una onda cuadrada que varía rápidamente entre 0V y 3.3V. La **potencia media** que la carga recibe depende del ciclo de trabajo.
- Aunque la salida es digital (0 o 1), la carga percibe un "efecto" analógico debido a la modulación, como en el control de brillo de un LED.

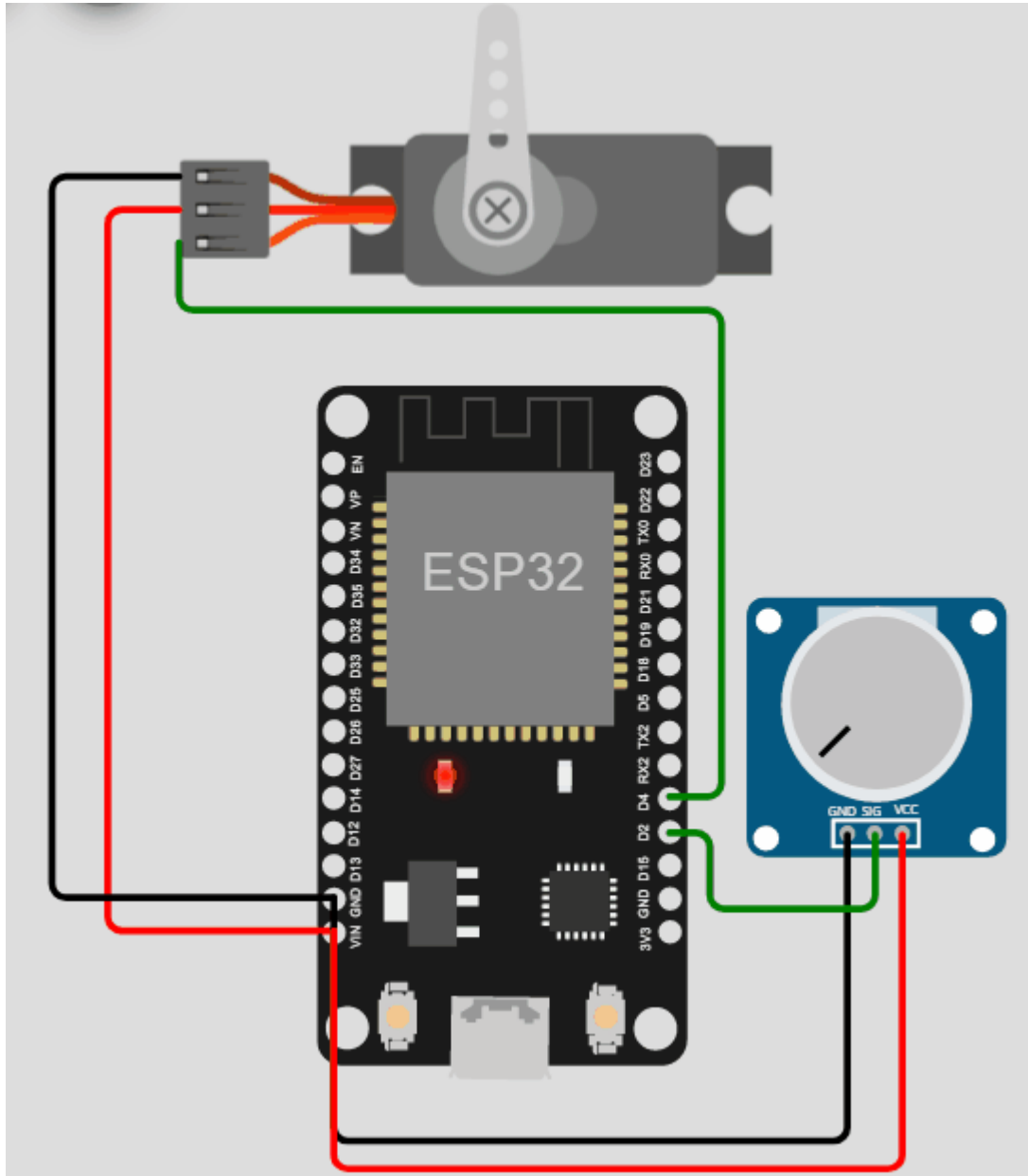
Los DAC del ESP32 tienen una resolución de 16 bits, 65.536 niveles

https://www.esploradores.com/micropython_adc/




0
0.5
1.0
1.5
2.0 ms


makeagif.com

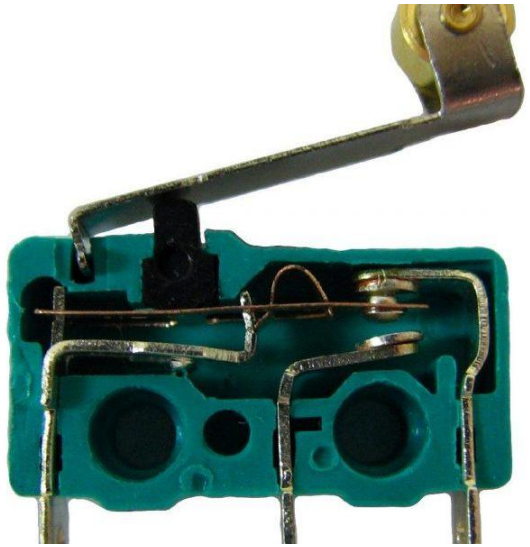

ESP32
EN
VP
VN
D34
D35
D32
D33
D25
D26
D27
D14
D12
D13
GND
VIN
D23
D22
TX0
RX0
D21
D19
D18
D5
TX2
RX2
D4
D2
D15
GND
3V3
GND
SIG
VCC

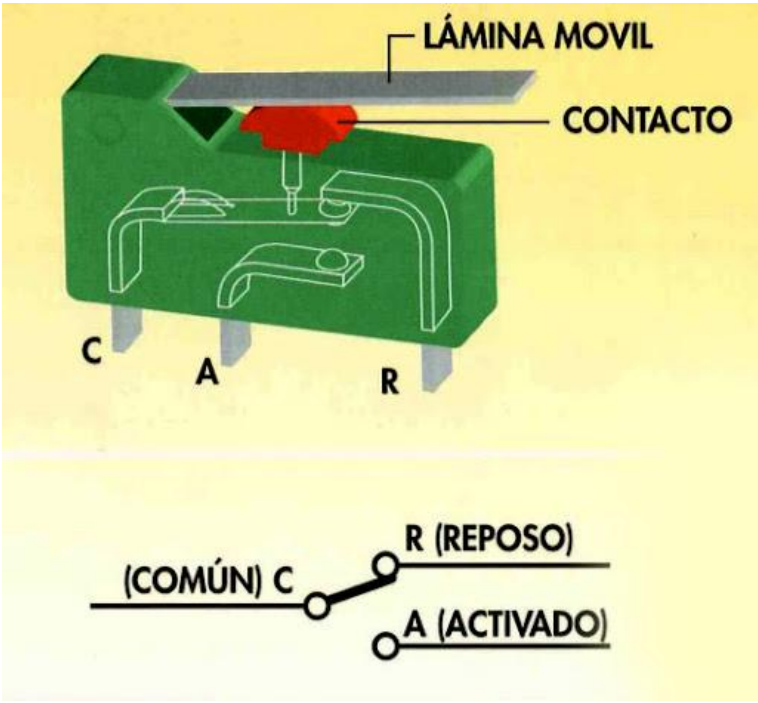
SENSORES DE CONTACTO



Los interruptores final de carrera son un interruptor (switch) convencional cuya única diferencia con éste último radica en dónde se lo coloca.

Un final de carrera se lo coloca, justamente, al final de un desplazamiento mecánico, y antes que se active el último interruptor que es la parada de emergencia



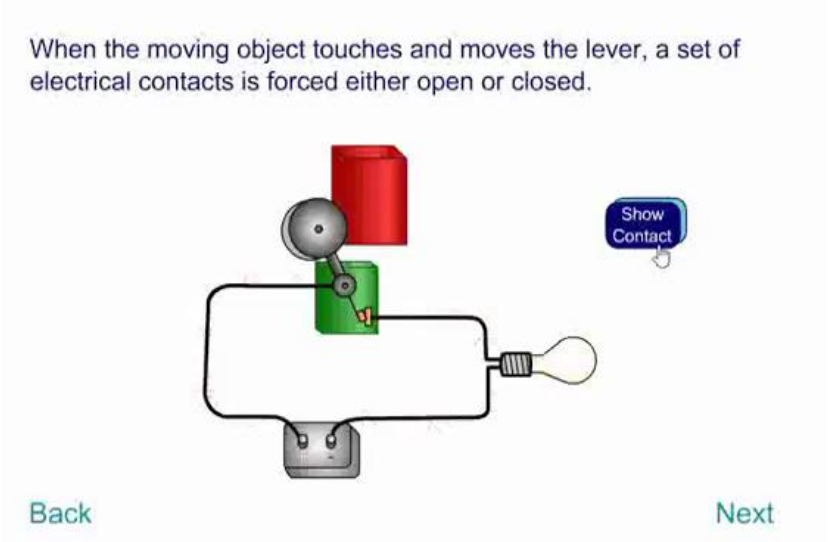


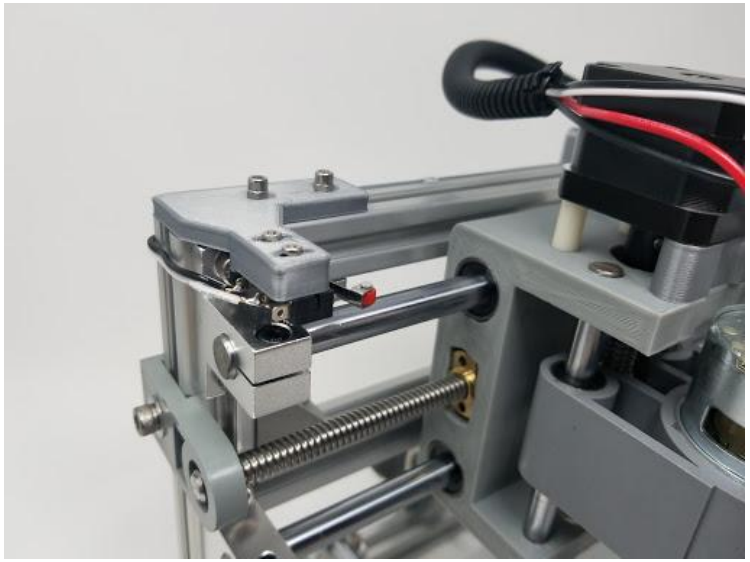
Los SENSORES DE FIN DE CARRERA , son sensores de contacto que muestran una señal eléctrica, ante la presencia de un movimiento mecánico.

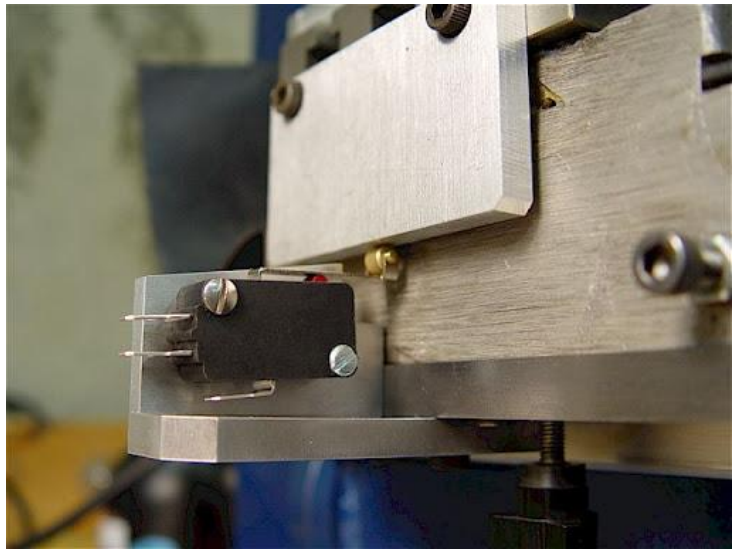
Son utilizados ampliamente en ambientes industriales para avisar de la presencia de objetos en una posición específica.

Se utilizan en diversas aplicaciones: Pueden determinar la presencia, ausencia, paso y posicionamiento de un objeto.

En un comienzo se los utilizaba para definir el final del recorrido de un objeto, de ahí que se llamen “interruptores de final de carrera”







- Internamente pueden contener interruptores normalmente abiertos (NA o NO en inglés), cerrados (NC) o conmutadores dependiendo de la operación que cumplan al ser accionados.

Switching type	COM — NC / NO
Normally closed type	COM — NC
Normally open type	COM — NO

Terminal symbols
COM: Common terminal
NC: Normally closed terminal
NO: Normally open terminal

Existen los de lengüeta, bisagra, palanca con rodillo, varilla, palanca metálica con muelle, de pulsador, etc.



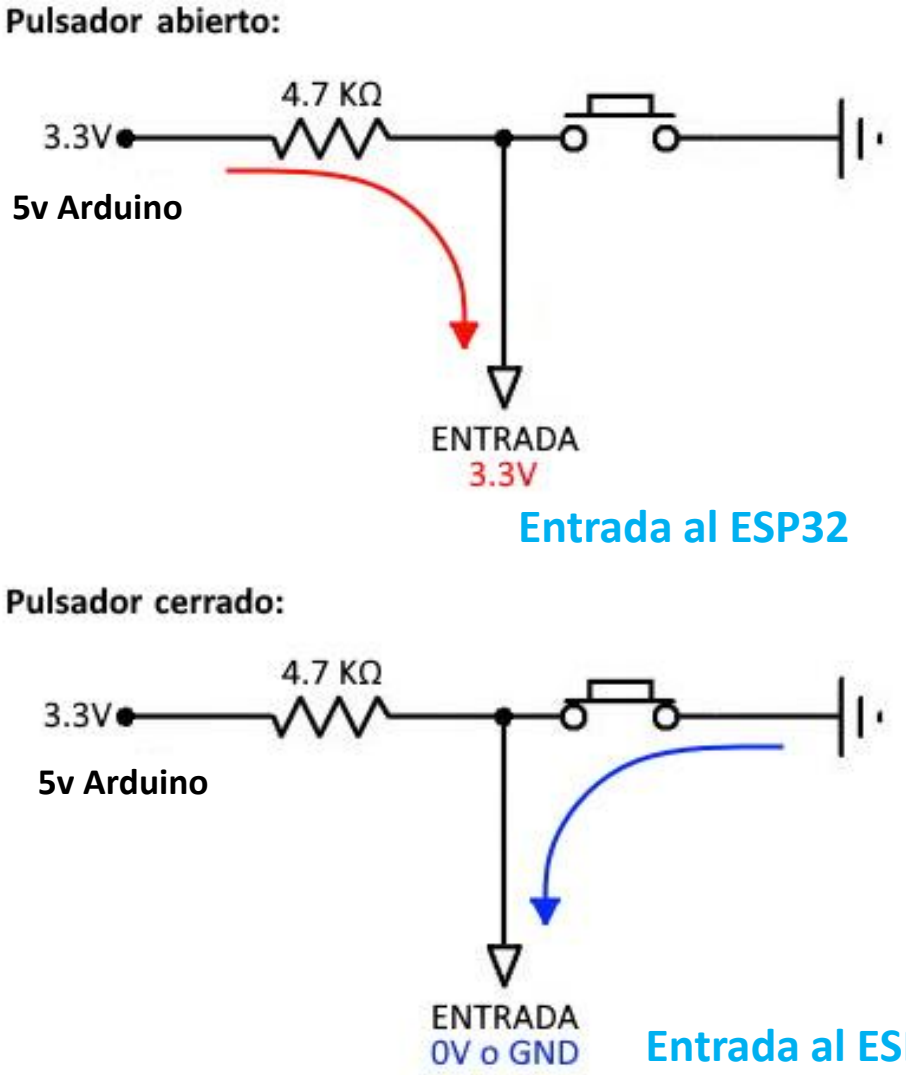
PULL-UP

Pull-up es como un resorte que siempre intenta "empujar" la entrada hacia arriba, hacia un estado alto .

Cuando el botón NO está presionado, la resistencia "tira" de la señal hacia el valor alto, asegurando que la entrada lea un 1 lógico.

Cuando se presiona el botón, conectas la entrada directamente a 0V (GND), lo que vence al pull-up, y la lectura cambia a 0 (bajo) lógico.

https://programarfacil.com/blog/arduino-blog/resistencia-pull-up-y-pull-down/



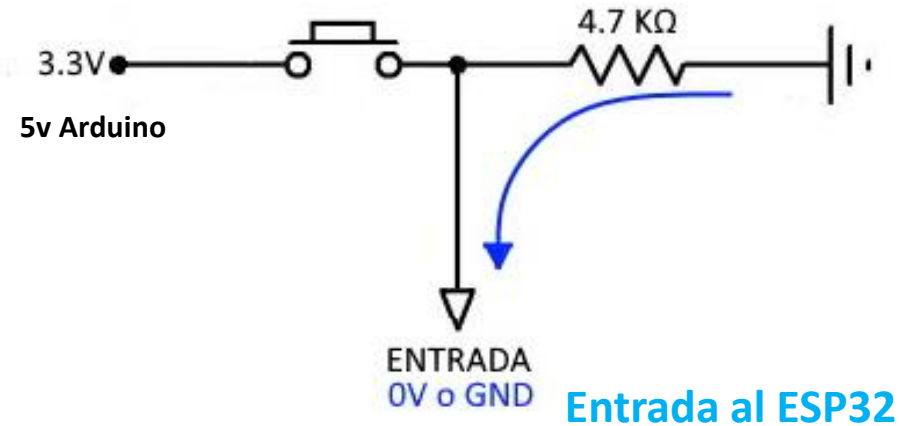
PULL-DOWN

Pull-down es como un peso que siempre intenta “tirar" la entrada hacia abajo, hacia un estado bajo (0).

Cuando el botón está ABIERTO, la resistencia “tirará" la señal hacia el valor bajo, asegurando que la entrada lea un 0 lógico.

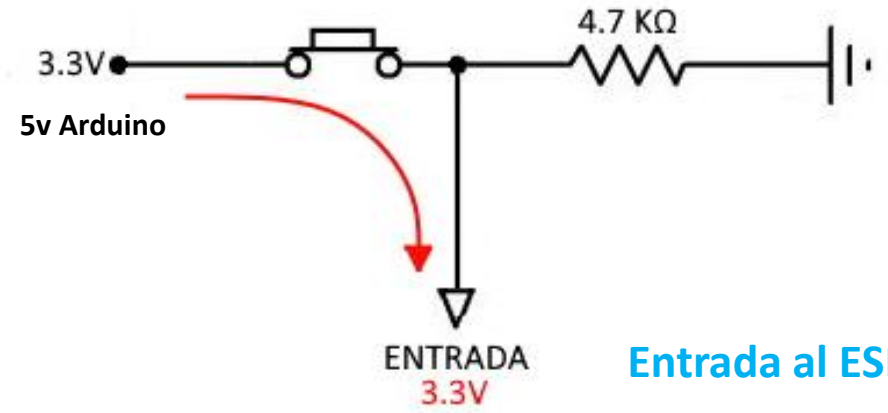
Cuando presionas el botón, conectas la entrada a un voltaje positivo (3.3V), lo que vence al pull-down, y la lectura cambia a 1 lógico.

Pulsador abierto:



Entrada al ESP32

Pulsador cerrado:



Entrada al ESP32


GRACIAS